

Peer-to-Peer Distributed Gaming Backend with Rollback

Alex Gao A17600181, Bran Zhang A16986434, Jason Zhu A69042759

Abstract

In this project, we explore various peer-to-peer distributed gaming backends. The goal is to present a consistent view to every client while maintaining a peer-to-peer backend. We use a simple classic 2D Gold Miner game to demonstrate the different backends. We provide visual comparison of game experience between authoritative host, lockstep, and rollback with different amounts of injected delay. Our implementation is publicly available at <https://github.com/kaijia2022/223B-Final-Project>.

Section 1: Introduction

1.1 Background

In recent years, many different types of games have become popular, with many of them being online real-time multiplayer games. Multiplayer games naturally face the distributed system problem of presenting a consistent view, and real-time games are required to present a consistent view within a short amount of time and withstanding reasonable network delay.

However, depending on the game genre and the delay assumptions, different performance guarantees or degrees of consistency are required. It is up to the game designer to decide what type of backend to run and whether to trade off the level of consistency between clients for performance. Online party games such as the one we explore in this project, are ideal for peer-to-peer backend: it is not required to run at very high frames per second (30 FPS is more than enough) for good game experience; Even though it is a real-time game, it does not require strict consistency – the consistency assumptions can also be relaxed. Running a peer-to-peer backend is financially beneficial as it saves the cost of renting a server; It is also beneficial performance-wise: if I am playing with my friends in San Diego, it makes no sense for them to rent servers hosted in LA to distribute their game packets; It also provides better availability since as long as we have working client machines and network, we can play.

1.2 Environment setup

We select Windows as the target gaming platform for fast prototyping and ease of testing. We use Hamachi VPN to provide a peer-to-peer network across all game clients, and we use the Raylib engine to build a lightweight gaming frontend to demonstrate our backend capabilities. To enable more controlled testing, we provide a set of delay injection APIs within our game binaries to simulate arbitrary outbound network delay within a certain range.

1.3 Peer-to-peer Approaches

Authoritative Host Approach

The host is responsible for resolving all the game states authoritatively. Each client that connects to the game only needs to continuously send their actions to the game host, and wait for the host to dispatch resolved game states. This host-relay approach certainly provides high performance and very strict consistency in terms of game states. However, this approach results in a single point of failure — if the host crashes, the game crashes for every connected client. Moreover, geographical proximity still matters: the closer a client is to the host, the better performance it will enjoy.

Lockstep Approach

The host acts just like every connected client: it runs the exact same code as the clients. Every client broadcasts to every other client and every client resolves the game state locally. No client renders the next frame until it receives the actions from every other client, processes the game states locally and renders the current frame. This approach still provides strong consistency but at the cost of poor performance for everyone, especially in the case where one client is suffering from significant outbound network delay.

Rollback Approach

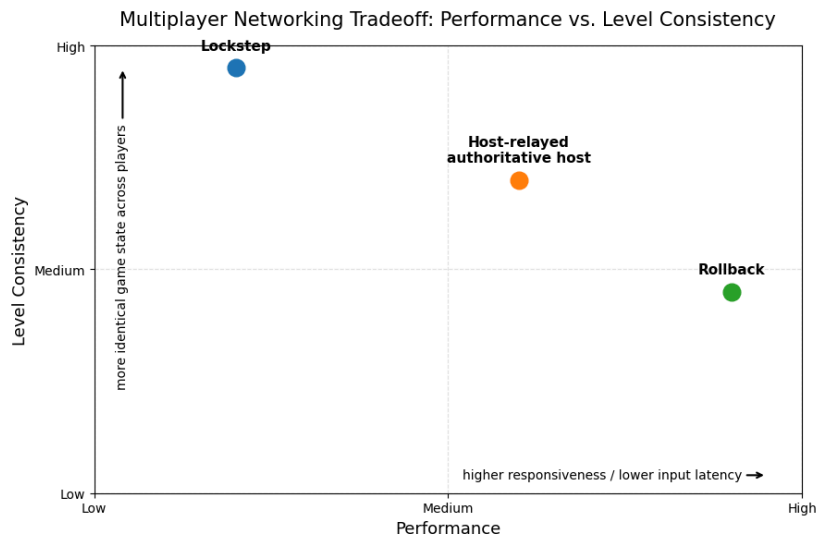
This approach improves on the lockstep approach by allowing a client to move onto the next frame even if it did not receive the full actions from every other client in the current frame. A client has to later “rollback” in time by replaying the missing client’s actions once they are received. Strict consistency is no longer guaranteed; different clients could see different views at a given moment in time. However, eventual consistency is still guaranteed, and performance is significantly improved on network delays compared to the lockstep approach.

The tradeoffs of the three networking approaches are summarized in Figure 1, where the x-axis represents performance and the y-axis represents consistency level.

Section 2:

Implementation

To display the three different synchronization models, we created a small game acting as a visual frontend. The game is a 2-4 player game, where each player is represented by a colored square on the screen. The goal of each player is to move towards gold coins placed randomly on the screen. The goal of each player is to move towards gold coins placed randomly on the screen, where upon making contact with a coin, the coin will relocate randomly on screen and the player that collected the coin will be awarded an increase in score of 10 points.



The first model implemented was a centralized authoritative server. Because the games were still peer-to-peer games, one player would be the “host” while other players would be “clients”. The host would be the centralized server, where it makes all the updates to the game state. Each client would send their input packets every frame to the host, and only update their screens upon receiving a response. The response would be a packet containing enough information to recreate the game state. This would allow for a client with incoming packet loss to still reconstruct the game state. This is the most common model seen in use for online multiplayer games, as it scales well to large numbers of players and supports the use of lower bandwidth protocols like UDP. However, this model does lead to a better experience for the host compared to the clients. While everyone's inputs will eventually be read, the host gets to react and give inputs immediately, while the clients must wait for the network round trip time to react to the updated state. In large multiplayer games where the central server is hosted by the developers/publishers, this host advantage doesn't exist, as everyone connects to the same server.

The second model is a peer-to-peer lockstep model. Instead of having a “host” and “client”, where each client establishes a connection with the host and the host processes and decides the state, each player will connect to each other and send their inputs to all other players. These inputs are then taken and locally processed to update the game state for each respective player. By making players send packets to all other players, we have increased network traffic, as now outbound packets have to be sent to all other players instead of just to one singular central server. To counteract this fact, the only response that a receiving player ever sends is just an acknowledgement of a received packet (handled by TCP). Each packet can also be minimal in its contents: a simple serialized C struct merely denoting which frame the input is for, and what the

input actually was. With this information, once players have received all input packets for that frame, they can then all locally update the game state for that frame according to those inputs. If every player is running the same game logic code then there will always be a consistent view of the game state presented to each player.

The third model we implemented was a relaxation of the peer-to-peer lockstep model. This version implemented network rollback. For lockstep, at each frame, every player would have to wait until it received the inputs for all other players, lowering performance and making the experience very choppy and slow. The idea behind rollback is that an individual player can update their local game state while waiting for inputs to arrive from other players, while it is waiting we can predict what the other players would have done in their movement, with our simple prediction just repeating their last received input. Each player stores the last known state that they have received all players' inputs for. Whenever a player receives input for a frame they have already simulated and predicted an input on, that player replaces the simulated input with the real input and resimulates from the last known state. The last known state is then advanced when the player receives all inputs for that state. The states are stored in a circular buffer to keep memory management minimal, as otherwise the game would take up too much memory if it were to run for long enough. The circular buffer serves as a natural garbage collection, as any old states will be overwritten. However, the circular buffer also means that there must be a maximum amount of predicting that can be done, and thus whenever the buffer is full, the player must wait until it receives the inputs required to advance the state and allow the buffer to be overwritten.

Lastly, there is the issue of randomness. Because in the lockstep and rollback approaches we only send inputs, spawning coins in the same location for all players becomes an issue. In the authoritative server, because we were sending game state responses, only the host needed to randomly assign coin locations and then tell other players where those coins ended up, but in lockstep and rollback, each player needed to simulate and come to the same random conclusion. Especially in rollback, it is possible for a predictive simulation to be wrong, and therefore the rollback must be able to generate the actual correct random positions of the coins. To solve this problem, we utilized a hash of the frame, colliding player's ID, as well as the coin and how many times that coin had been collided with to create replicable randomness as each player would only need the same hashing algorithm in order to be able to create the same simulations. Because it's based on what is currently happening in the state, on rollback, two independent players will arrive and see the same state at that frame.

Section 3: Results

The following three figures are GIFs demonstrating the differences between a central authoritative server, lockstep, and rollback from top to bottom. Each figure shows a player on the

left and a player on the right, with the first two figures displaying the right side player (the blue square) providing input. For testing, we put random inputs and injected random amounts of delay trying to create a divergence. Early implementations of rollback did find many cases of divergence in presented game states, however the current implementation should be reasonably correct.

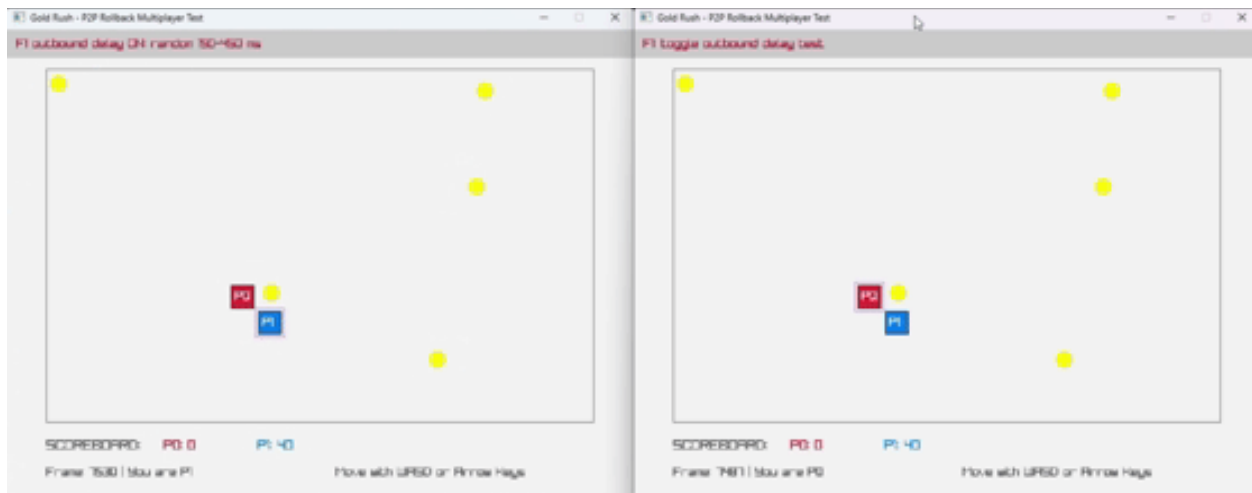
In the first figure we can see a demonstration of the authoritative server. The left player is the host while the right player is the client. In it, we can see that while the right player is the one providing input, the update is actually first seen on the left player's screen before the right player's. This is due to the nature of the central server implementation described in the previous section. The inputs on the right side also feel extremely delayed when network delay is injected, as the right side player now has to wait for the packet to arrive at the host and return with the game state in order to update while the left side player's inputs feel smooth as they are directly handled.



The second figure shows the lockstep implementation, where again the right player is the only one providing input. We can see that in the lockstep, the right side player moves at the same time on both the screen for the left player and right player. As opposed to the authoritative server, a delay on one side ends up causing a decrease in performance for both players.



The third and final figure shows the implementation of rollback. Here we can see that the left side has run ahead and is exhibiting a delayed output. The GIF starts with the left player moving first, followed by the right player moving. In the frame timeline, the right player actually collects the coin, while the left player in wall-clock time collects the coin first. This results in the left player initially increasing their score, but as soon as the right player's inputs arrive, it executes the rollback, updating the score to reflect what really happened from the perspective of what happened on each frame. Also of note, until the right player gets to the frame in which the left player moves, it never actually updates the left player's position on the right player's screen, however the left player updates the right player's position almost immediately, as it receives them. As opposed to the authoritative server and the lockstep implementation, both players can smoothly move on their screens. This leads to a smoother gameplay experience, with the occasional change in game state as a result of rollback.



Section 4: Conclusion

In summary, we implemented and evaluated three different approaches to peer-to-peer distributed gaming backends for real-time party games with different injected delays. There is no single best approach, but one might outperform the others depending on the specific performance and consistency assumption. For games that require a strict consistency across players with the most fairness, lockstep would be the better choice, with a relaxation of fairness making a central server be the better choice and a relaxation of consistency making rollback the better choice. Ultimately, it is up to the game backend designers to select the approach that best fits the performance and consistency requirements of their game.